



Agent Quality & Token Optimization



JASON COOK

Principal Solution Engineer – Global Black Belt

Microsoft

What we'll cover today

- 01 Why agent quality is the better focus to optimize tokens
- 02 Foundational Knowledge of LLMs, Agents & Context Windows
- 03 Quality and Token Controls & Practical Optimization Tips



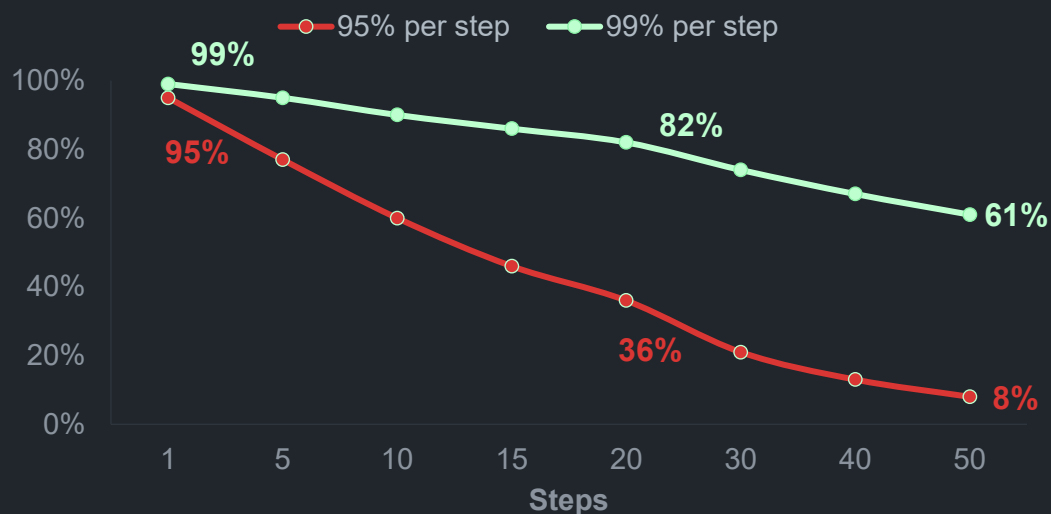
Agent gambling is no longer sustainable

When tokens are cheap, agent accuracy is not important. Once they are not, they require engineering work.



The Compound Error Problem

Even at 99% per step, a 50-step workflow only lands at 60%.



ZOOMING OUT

Chances of an Agent miss compound quickly

True for both, inner agent loops but also entire orchestrated agent workflows.



Instead of
counting tokens,
make every
token count!



Quality & Token Controls



Where quality gets lost and tokens get burned

Wrong model for the job Using a high-multiplier reasoning model (Opus) for routine edits a mid-tier model handles at a fraction of the cost.	Not clearing sessions Not clearing context between unrelated tasks or modes. History carries forward until compaction, and compaction itself costs tokens.	Not splitting tasks Trying to single shot complex tasks leads to unnecessary context being carried in between stages. Split your work into small chunks and sequentialize & parallelize where possible.
Imprecise prompts Asking Copilot to 'look at src/' when only one function matters. A single large file can consume 20k+ tokens.	Missing Guardrails Missing guardrails like tests, linters, security scans prevents agents from early auto-correction, and leads to expensive debugging- and fixing sessions later.	Oversized instructions .github/copilot-instructions.md injected on every turn. Treat it as high-cost configuration, not documentation.

The two biggest levers for optimizing quality & tokens



Model choice & auto mode

Reasoning models (Opus, GPT-5.5)
for sync tasks like planning, architecture, debugging.

Mid-tier models (Sonnet, GPT5.4)
for async implementation.

Low-tier (Haiku, GPT-mini)
for small refactorings, repetitive tasks, doc updates.

Auto Mode as the lazy default.



Provide only relevant context

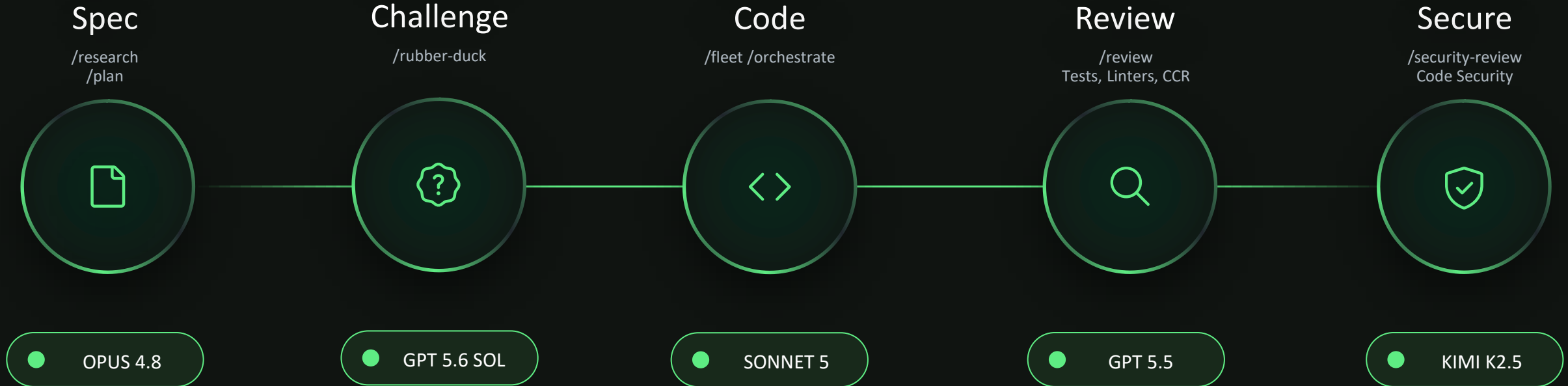
As much as required, as little as necessary.

Context Engineering as your guidance and upskill potential.

Compacting sessions can be helpful – but they can also lead to valuable information being lost. Used it cautiously.

Use /clear often, for each new task.

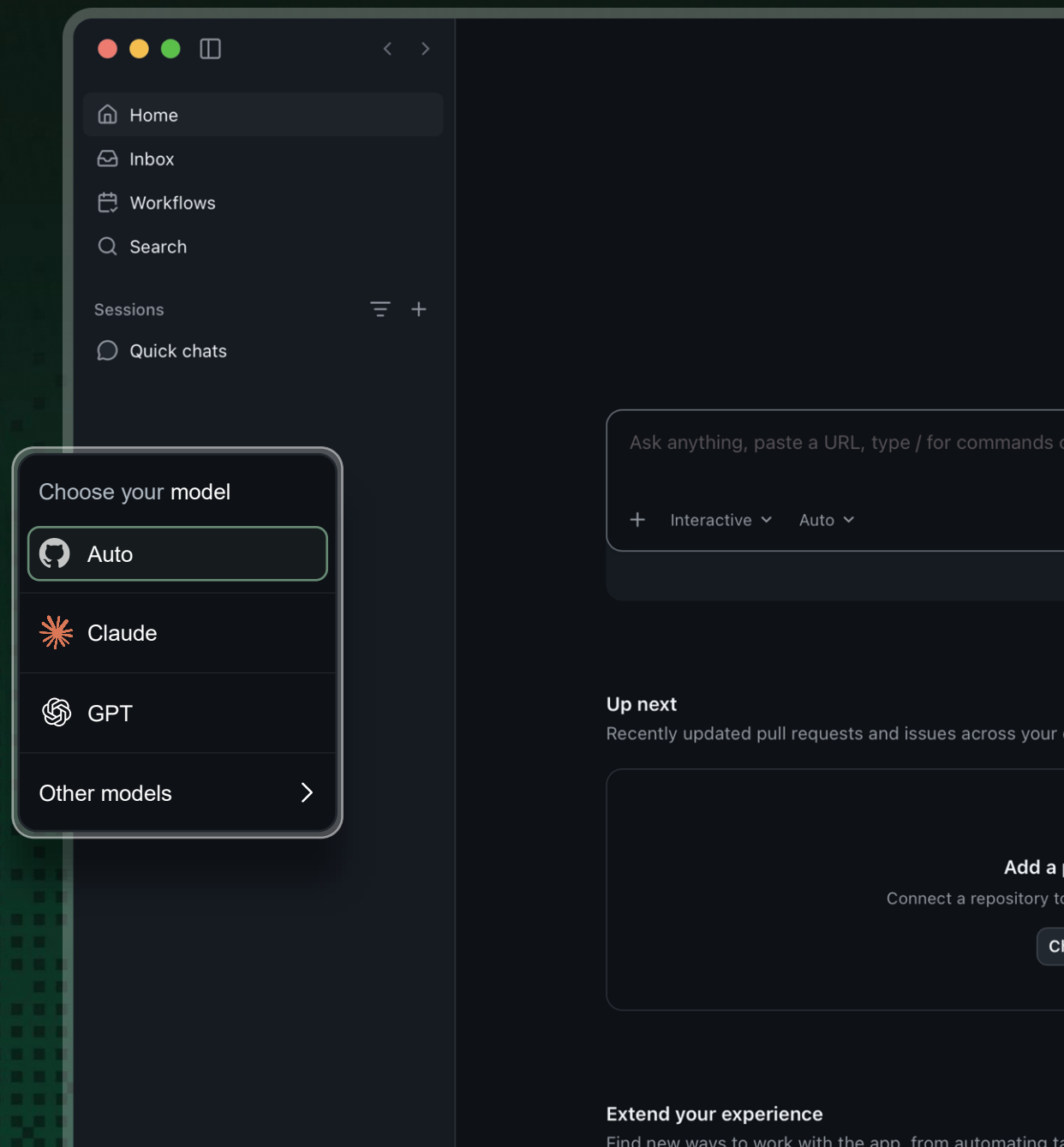
The right Mode with the right Model and the right Context.



Enhanced automodel picker: Auto v2

Our goal: better fit, less manual tuning, more predictable quality.

- Broader model access for richer tasks and premium models
- Tiered auto choices to align quality, cost, and speed
- Token-aware routing helps keep complex sessions efficient
- Better handling of long-running and multi-turn work to reduce drift



Your Prompt

Be precise. Short and Sharp descriptions.	Add an imperative goal Tell the agent what to do	Add known context Files, Folders, Websites, etc.
---	--	--



Example of prompt discipline

Shorter, sharper prompts don't just save input tokens; they cut the volume of output and tool calls.



Expensive

\$ copilot

"Look at my repo and figure out why users are getting 500 errors on login. Check the auth code, database layer, middleware, and logs. Fix it."

Scans many files, reads large chunks, burns tokens before narrowing down.



Efficient

\$ copilot

"In @src/auth/login.ts, method handleLogin returns 500 when email has unicode chars. Propose a fix."

Targets one file, one function, one case.

DIVIDE AND CONQUER YOUR WORK

Research → Plan → Implement



Having the right Guardrails prevents Agent misses early



Tests

Tests are the backbone of your programs – they guide agents to implement the right behavior and ensure regressions are detected early.



Good Architecture

Clarifying to the agent what goes where increases the chances of success and reduces tokens wasted in search.



Linters

Consistent formatting makes codebases easier to read and reason about – also for agents.



Quality & Security Scans

Providing feedback about Quality & Security as early as possible ensures no rework is necessary

Agent Configs

- | | |
|---------------------------|---|
| ✓ Persistent instructions | COPILOT-INSTRUCTIONS.MD |
| ✓ Custom Agents. | ./GITHUB/AGENTS/*.AGENT.MD |
| ✓ Skills | ./GITHUB/SKILLS/*/SKILL.MD |
| ✓ MCP | |
| ✓ Subagents | |
| ✓ Scoped instructions. | ./GITHUB/INSTRUCTIONS/*.INSTRUCTIONS.MD |
| ✓ Prompt Files | ./GITHUB/PROMPTS/*.PROMPT.MD |
| ✓ Copilot Memory | |

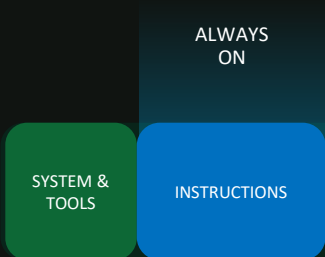
ENSURE CONSISTENCY AND QUALITY

Persistent Instructions

🔒 `./.GITHUB/COPILOT-INSTRUCTIONS.MD`

🔒 `./AGENT.MD`

- ✓ Put in:
 - the non-negotiables of your projects
 - log reoccurring agent misses
 - Statements to trim output (“be concise”)
- ✓ Keep them very small.
- ✓ Don’t use AI to generate them.
- ✓ Iterate, maintain and even recreate them often.



Example of good instructions

COPILOT-INSTRUCTIONS.MD IS RE-SENT ON EVERY TURN. EVERY LINE IS A RECURRING TAX.

`.GITHUB/COPILOT-INSTRUCTIONS.MD`

`## Project`

`Java Spring Boot microservices (user, order, payment)`

`## Standards`

- `- Constructor injection only`
- `- Return ResponseEntity from controllers`
- `- GlobalExceptionHandler via @ControllerAdvice`

`## Do not suggest`

- `- Changes to pom.xml or application-*.yaml`
- `- Anything in /target or /build`



KEEP

- Standards the model can't infer from code
- Explicit 'do not' rules for files/patterns
- One-line tech-stack context

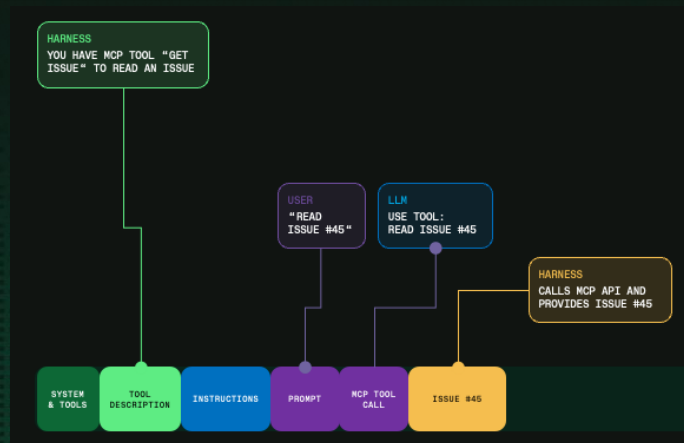


CUT

- Onboarding essays and team history
- Architecture diagrams as ASCII art
- Full style guides — link instead
- Anything redundant with the code itself



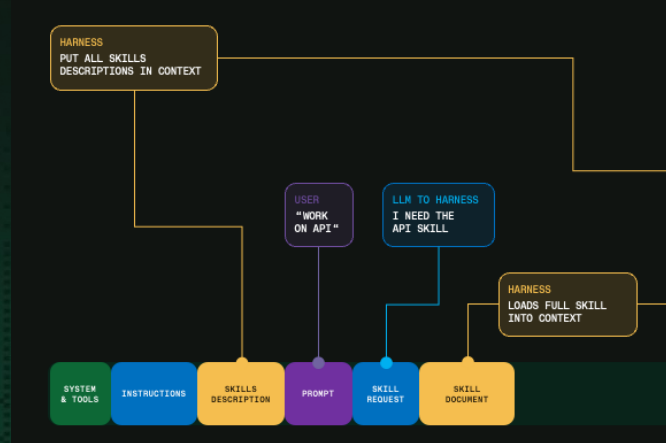
Agent Configs



DELIVER STARTING POINTS

Custom Agents

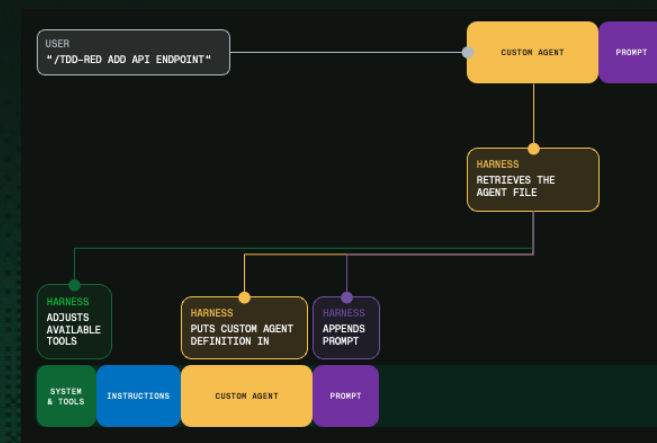
Design recurring workflows & behaviors for your agents.



ADD ON DEMAND CONTEXT

Skills

Add on-demand context with static knowledge in your Codebase.



INTEGRATE 3RD PARTY TOOLS

MCP

Add on-demand context with dynamic information retrieval - ideal to integrate external systems.

Observe and adjust

Analyze current context & cost AI Credit consumption is surfaced in each harnesses at the end of each prompt execution: - Copilot App: /context & Insights - CLI: /context & /usage - VS Code: Context View & Agent Debug Logs Use /chronicle to look at session logs and analyze them for optimization.	Analyze usage & behavior Use /chronicle (available in all harnesses) to query your overall usage. Use `/chronicle tips` to get tips scoped to your behavior.	Higher Level Metrics Higher level metrics mostly have to be set up / given access to by your enterprise admins. There is different methods: - Otel: VSCode offers open telemetry options - GitHub Impact Dashboards: Shipped in July, these dashboards help in assessing overall impact and ROI of agents - Agent Session Streaming (Private Preview)
--	---	--

Power User Guidance

These Tips require good **knowledge and time-invest**, as they are conditional and / or come with trade-offs.

Think in Code

Prefer creating scripts to analyze files over feeding it to AI.

Consider CLIs vs. MCPs

CLI Tools can be more optimal in certain scenarios due to less static tokens.

Improve Shell Outputs

Shell outputs can be extremely long – use something like <https://github.com/rtk-ai/rtk> to reduce their outputs.

Run “/chronicle tip” regularly

Analyze your usage in Copilot CLI to find improvement areas.

Collapse Tool Calls

Using something like <https://github.com/jsturtevant/copilot-codeact-plugin> can help collapse multiple tool calls into one.

Model Specific Context Optimization

Models behave different and can be tweaked.





Thank you